CPE334 — Lab 1 · Git & GitHub Cheat Sheet

TokTickIT · everything you need for the Lab 1 workflow · keep this open beside your editor

0 · One-time setup

Check Git is installed, then set your identity (used on every commit):

```
git --version
git config --global user.name  "Your Name"
git config --global user.email "you@kmutt.ac.th"
```

Sign in to GitHub. Easiest is the GitHub CLI:

```
gh auth login        # follow the browser prompts
```

IMPORTANT For HTTPS pushes, your GitHub account password will NOT work. Use gh auth login, or a Personal Access Token (Settings → Developer settings → Tokens) when Git asks for a password.

1 · The Lab 1 branch model

```
main                        ← stable release (protected)
 └─ lab1-staging            ← integration branch
      ├─ feature/1-project-foundation
      ├─ feature/2-health-check
      ├─ feature/3-category-seed
      └─ feature/4-category-list
```

Rule: never commit directly to main or lab1-staging. Each Issue is built on its own feature branch → Pull Request into lab1-staging. After all four are merged, open ONE release PR from lab1-staging → main.

2 · Create the repository & branches

Option A — website + clone

On GitHub: New repository → name it toktickit → Create. Then locally:

```
git clone https://github.com/<you>/toktickit.git
cd toktickit
git checkout main
git checkout -b lab1-staging
git push -u origin lab1-staging
```

Option B — GitHub CLI

```
gh repo create toktickit --private --clone
cd toktickit
git checkout -b lab1-staging
git push -u origin lab1-staging
```

TIP Add your peer reviewer as a Collaborator (Settings → Collaborators → Add people) so they can review and approve your PRs.

3 · Feature-branch workflow (repeat for each Issue)

1) Always branch from an up-to-date main:

```
git checkout main
git pull
git checkout -b feature/2-health-check
```

2) Work, then stage and commit:

```
git status                 # see what changed
git add .                  # stage everything (or: git add <file>)
git commit -m "feat: add /api/health endpoint"
```

3) Push (first time sets the upstream):

```
git push -u origin feature/2-health-check
# later commits on the same branch:
git push
```

TIP Commit messages: short, imperative, prefixed — feat: fix: test: docs: chore:. Commit small and often.

4 · GitHub Project board (Kanban)

Create: repo → Projects → New project → Board. Add a Status single-select field with EXACTLY these options, in order:

```
Backlog · Specified · Started · PR Review · Fixing · Done
```

Create the four Issues (Issues → New issue), then add each to the board starting in Backlog. Move a card by dragging it or changing its Status.

Status	When to use it
Backlog	Issue created, not yet reviewed/understood
Specified	You understand it and it is ready to build
Started	Feature branch created, work in progress
PR Review	PR is open and your partner is reviewing
Fixing	Review requested changes / tests failed
Done	PR approved, tests pass, merged to lab1-staging

5 · Open a Pull Request

```
gh pr create --base lab1-staging --head feature/2-health-check --fill
# or on the website: "Compare & pull request"
```

IMPORTANT Set the PR base branch to lab1-staging. GitHub defaults the base to main — change it, or your feature merges straight to main.

Add your partner under Reviewers. In the PR description, link the Issue with "Closes #2" so it auto-closes on merge.

6 · Peer review (both directions)

- **As reviewer:** PR → Files changed → Review changes → Approve or Request changes, with a comment.
- **As author:** read the comments, push fixes to the SAME branch (they appear in the PR automatically), reply, then re-request review.

Merge only after approval AND passing tests: Merge pull request → Confirm → Delete branch.

7 · Release to main

When all four features are in lab1-staging:

```
gh pr create --base main --head lab1-staging --title "Lab 1 release" --fill
```

Review, confirm tests pass, then merge. main now holds the finished Lab 1.

8 · Merge conflicts (quick fix)

```
git checkout feature/xyz
git pull origin lab1-staging      # bring in latest
# edit the marked  <<<<<<<  =======  >>>>>>>  sections, keep the right code
git add .
git commit -m "merge: resolve conflicts with lab1-staging"
git push
```

9 · Undo & fix common mistakes

Situation	Command
Unstage a file	`git restore --staged <file>`
Discard local changes to a file	`git restore <file>`
Fix the last commit message	`git commit --amend -m "new message"`
See history as a graph	`git log --oneline --graph --all`
Switch branches	`git switch <branch>` (or: `git checkout <branch>`)
Committed node_modules or .env	add to .gitignore, then: `git rm -r --cached node_modules .env` → commit
Update your branch with latest staging	`git pull origin lab1-staging`

10 · Never commit secrets

.gitignore must include:

```
node_modules/
.env
dist/
build/
```

Commit .env.example (a blank template). Never commit the real .env.

Quick command reference

Command	What it does
`git clone <url>`	Copy a GitHub repo to your machine
`git status`	Show changed / staged files
`git add <file> / git add .`	Stage changes for the next commit
`git commit -m "msg"`	Save staged changes as a commit
`git push / git push -u origin <branch>`	Upload commits (-u sets upstream first time)
`git pull`	Download and merge the latest from GitHub
`git checkout -b <branch>`	Create and switch to a new branch
`git switch <branch>`	Switch to an existing branch
`git branch`	List local branches
`git log --oneline --graph --all`	Compact visual history
`git restore <file>`	Discard local changes to a file
`git restore --staged <file>`	Unstage a file
`git merge / git pull origin <branch>`	Bring another branch's changes in
`gh repo create <name> --private --clone`	Create + clone a repo (CLI)
`gh pr create --base <b> --head <b> --fill`	Open a Pull Request (CLI)
`gh pr status`	See your PRs and review requests

Lab 1 Part 1 submission — Git evidence checklist

- Repository URL, GitHub Project URL, all 4 Issue URLs, and all Pull Request URLs.
- Project board with all four Issues in Done.
- Commit history showing feature branches → lab1-staging → main.
- Rendered README.md and .gitignore.
- docs/lab-01/reviewer.md with partner approvals and a real review comment + response, both directions.